

module - 4

1. Introduction to algorithm analysis

- Algorithm analysis is the process of measuring the efficiency of an algorithm based on the time it takes to execute and the memory it uses.

Time complexity → Execution time

Space complexity → Memory usage.

2. Time complexity

Time complexity tells how the running time of an algorithm increases as the input size (n) increases.

```
for (int i=0; i<n; i++)  
{  
    cout << i;  
}
```

Time complexity : $O(n)$.

3. Space complexity

Space complexity is the amount of memory required by an algorithm during execution.

Formula:- Space complexity = Fixed space + Auxiliary space

Example:- `int arr[100];`

Space complexity = $O(1)$

4. Asymptotic Notations

a) Big $O(n)$

- Represents the upper bound (worst case)
- Maximum time an algorithm can take.

Ex:- linear search $\rightarrow O(n)$

b) Omega (Ω)

- Represents the lower bound (best case)
- Minimum time an algorithm can take

Ex:- linear search element at first position $\Omega(1)$

c) Theta (Θ)

- Represents the exact bound
- Best and worst case grow at the same rate

Ex:- $O(n)$

5. Best, average & worst case

Case	meaning
Best	Minimum execution time
Average	Average execution time
Worst	Maximum execution time.

Example:- Best = $O(1)$ • average = $O(n)$ • worst = $O(n)$

6. Rules to calculate complexity

1. Ignore constants
2. Keep the highest power
3. Sequential loop $\rightarrow$ Add complexities
4. Nested loops $\rightarrow$ Multiply complexities.

7. Time complexity of loops

Single loop

for (int i=0; i<n; i++) complexity : $O(n)$.

loop running fixed times

for (int i=0; i<10; i++) complexity = : $O(1)$.

8. Time complexity of loops.

```
for (int i=0; i<n; i++)  
{  
    for (int j=0; j<n; j++)  
    {  
    }  
}
```

complexity = $O(n^2)$.

9. Time complexity of logarithmic loops

when the loop variable is multiplied or divided by a constant.

Example :-

```
for (int i = 1; i < n; i = i * 2)
```

complexity = $O(\log n)$.

10. Space complexity basics.

★ Constant space `int a, b, c`

Space complexity = $O(1)$

★ Linear space `int arr[n];`

Space complexity : $O(n)$.